

CONCORD v0.3 Core Specification

Backend assessment and control framework for agent-safe systems

Status: Normative draft | Version: 0.3 | Family: CONCORD | Date: March 2026

Purpose: This document defines the normative runtime model for CONCORD. It specifies entities, required behaviors, consistency rules, authority controls, coordination semantics, recovery rules, and the advisory planning surface exposed to AI agents.

1. Scope and design principles

CONCORD is a backend assessment and control framework that makes applications safe for AI agents through explicit contracts, composable authority, bounded execution, portable consistency rules, safe concurrency, and recoverable operations.

- OperationContext is advisory; intent admission is authoritative for any mutating action.
- Projected reads may guide planning, but they do not authorize mutation unless explicitly marked authoritative.
- Overrides restrict by default; privilege expansion requires an explicit policy grant.
- Every conflict code must carry structured agent-action guidance.
- Every multi-resource mutation path must either declare saga coverage or be flagged as uncovered.

2. Layer model

Layer	Responsibility	Primary outputs
Contract layer	Resources, actions, schemas, business states, guards, compensation hooks	Action manifests, state manifests
Coordination layer	Leases, queues, contention rules, version checks, consistency semantics	Coordination state, admission decisions
Identity, trust, and budget layer	Sessions, intents, agent classes, capability sets, throttles, circuit breakers	Authority decisions, budget decisions
Recovery and observability layer	Receipts, event logs, saga state, compensation, timing, metrics	Receipts, event trails, recovery advice
Discovery and refactor layer	Codebase scan, readiness scoring, danger mapping, dependency analysis	Readiness report, danger map, patch plan

3. Entity definitions

3.1 AgentClass

AgentClass defines a trust tier and composes one or more capability sets. It is the normative policy anchor for an agent family.

Field	Type	Rule
agent_class_id	string	Unique identifier
name	string	Human-readable class name
trust_tier	enum	T0/observe, T1/propose, T2/bounded, T3/privileged, T4/governed_critical
capability_sets	array	One or more CapabilitySet references
budget_profile_id	string	Required budget profile
override_rules	object	Restrict-only by default
prod_allowed	boolean	Whether production operation is allowed
requires_human_gate_for	array	Action families or risk classes requiring approval

3.2 CapabilitySet

CapabilitySet is a composable permission bundle. It may grant actions and carry scoped exclusions.

Field	Type	Rule
capability_set_id	string	Unique identifier
allowed_action_families	array	Positive grants
restricted_resource_types	array	Explicit denials by resource type
lease_permissions	array	Lease classes that may be acquired
max_authority_mode	enum	Highest execution mode permitted
default_budget_profile_id	string	Optional default budget profile
exclusions	array	Scoped denials by action, tag, environment, or risk threshold

Override semantics: override_rules may narrow rights derived from composed capability sets. Expansion beyond composed capability sets requires an explicit PolicyExpansionGrant artifact approved by policy.

3.3 Session, Intent, Resource, Lease, Receipt, SagaRun

Entity	Minimum required fields	Notes
Session	session_id, agent_id, task_id, started_at, expires_at, mode, status	Carries traceability and mode restrictions
Intent	intent_id, session_id, resource_type, resource_id, action_name, idempotency_key, intent_status	Authoritative admission and execution unit
Resource	resource_id, resource_type, business_state, coordination_state, version	Separates workflow state from coordination state
Lease	lease_id, resource_id,	Short-lived ownership or

	lease_type, holder_session_id, granted_at, expires_at, status	reservation
Receipt	operation_id, intent_id, previous_state, next_state, result_status, duration_ms	Machine-verifiable action result
SagaRun	saga_id, root_intent_id, status, timeout_at, timeout_policy, compensation_status	Tracks multi-step commit and compensation

4. OperationContext contract

OperationContext is the composite planning surface for agents. It is advisory and time-bounded. It does not reserve rights, admit mutation, or supersede authoritative checks at intent admission.

Field	Required	Meaning
resource_id	Yes	Resource anchor
business_state	Yes	Workflow state
coordination_state	Yes	Current contention / lease / queue condition
allowed_actions	Yes	Actions that appear admissible at assembly time
blocked_actions	Yes	Actions currently blocked
active_leases	Yes	Current leases affecting the resource
pending_intents	Yes	Active or queued intents
budget_remaining	Yes	Remaining session or class budget summary
recommended_next_action	Yes	Planner hint for next safe move
context_assembled_at	Yes	Timestamp of assembled context
context_ttl_ms	Yes	Advisory freshness window
freshness_status	Yes	fresh, warning, stale
authoritative_for_mutation	Yes	Must be false

Normative rule: Agents may use OperationContext for planning and sequencing. Every mutating action must still pass authoritative intent admission checks, including version, lease, queue, policy, consistency, and budget evaluation.

5. Consistency profiles

Every action contract shall declare a consistency profile and the related required enforcement behavior.

Profile	Required behavior	Typical use
STRONG	Reads are authoritative. Mutations use compare-and-swap directly. Stale writes return STALE_VERSION.	Single-writer primary stores
SESSION_MONOTONIC	A session may not act from a view older than its own prior write watermark. Violations return SESSION_STALE_VIEW.	Long-running sessions with monotonic reads

READ_YOUR_WRITES	After a write, same-session reads must route to a fresh source or return READ_FRESHNESS_UNAVAILABLE.	Apps with primary/replica split
EVENTUAL	Projected reads may guide planning only. Every mutation requires mandatory authoritative recheck before commit.	Replica-backed planning surfaces
ASYNC_PROJECTION	Read responses include projection_timestamp, projection_lag_ms, and staleness_tolerance_ms. Mutations route to authoritative endpoints.	Read models and async projections

- Action contracts must carry: consistency_profile, authoritative_recheck_required, projection_tolerance_ms, and session_watermark_required where applicable.
- Projected version values shall not be treated as sole authorization for mutation under EVENTUAL or ASYNC_PROJECTION.

6. Budget enforcement model

Enforcement point	Checks	Authority
Gateway layer	Requests per minute, burst limit, concurrent request cap, open-circuit rejection	Preliminary only
Intent layer	Risk-weighted budget, mutating action quotas, per-resource-class quotas, compensation-rate throttles	Authoritative

If a request passes gateway throttles but fails intent-layer budget admission, the system shall return a specific budget dimension exceeded, along with cooldown guidance when available.

7. Conflict resolution and coordination

Each resource type shall declare a conflict_resolution_strategy. The default order is version check, lease check, queue admission, retry/backoff recommendation, policy arbitration, human escalation, then negotiation if enabled.

Strategy	Default behavior
default	General workflow systems
fail_fast	Version/lease then immediate reject
queue_first	Fairness-oriented review systems
lease_first	Shared-edit surfaces
policy_first	High-risk controlled domains
custom	Domain-specific order documented in contract

8. Saga and compensation rules

Field	Required behavior
timeout_policy	One of fixed, step_adaptive, heartbeat, external_gated
timeout_at	Hard deadline for fixed mode
step_timeout_ms	Per-step ceiling for step_adaptive mode
heartbeat_interval_ms	Required signal interval for heartbeat mode
external_dependency_timeout_ms	Upper bound on suspended wait in external_gated mode
max_compensation_attempts	Ceiling before poison

- When timeout is reached and the saga is not terminal, the saga shall transition to timed_out and begin compensation if compensable.
- When compensation attempts exceed max_compensation_attempts, the saga shall transition to poisoned, emit SAGA_POISONED, and require human intervention.
- Poisoned sagas shall not be automatically retried unless explicitly reopened by policy or human action.

9. Required observability

- Receipts shall include duration_ms.
- Coordination telemetry shall include lease contention rate, average queue wait, stale write rejection rate, budget throttle rate, compensation invocation rate, and circuit breaker trip count.
- All mutating paths shall emit a receipt and an event entry linking the acting session, intent, and resource version change.

10. Minimum implementation profile

1. Define resource and action contracts.
2. Separate business_state from coordination_state for mutable resources.
3. Support version-aware mutation and idempotency keys.
4. Implement Session, Intent, AgentClass, CapabilitySet, and BudgetProfile.
5. Expose OperationContext as an advisory planning endpoint.
6. Implement basic exclusive leases and structured receipts.
7. Define at least one consistency profile and one conflict strategy per mutable resource type.